

山东建筑大学 云计算与微服务 作业

班级：软件 233 学号：202302105024 姓名：赵冠杰 日期：2026.5.27

作业八 分布式事务 Seata

一、什么是分布式事务？

分布式事务是指在分布式系统架构中，一个业务操作需要跨多个独立的数据库或服务节点完成，且这些操作必须同时成功或同时失败，以保证数据一致性。

在单体应用中，事务由本地数据库的事务管理器（如 MySQL 的 InnoDB）统一管理，通过 ACID 特性保证数据一致性。但在微服务架构中，每个服务拥有独立的数据库，一个业务请求可能涉及订单服务、库存服务、支付服务等多个数据库的写操作，无法依靠单一数据库的事务机制。

核心挑战：

网络不可靠：服务间通过网络通信，可能发生延迟、超时、断连，导致部分操作成功、部分失败。

数据分散：数据分布在多个独立的数据库中，无法用同一个事务管理器统一协调。

高并发场景：分布式环境下，并发控制和锁机制变得更加复杂。

常见的分布式事务解决方案包括：两阶段提交（2PC）、三阶段提交（3PC）、TCC（Try-Confirm-Cancel）、Saga 模式、AT 模式、XA 模式等。Seata 是阿里巴巴开源的分布式事务解决方案，提供了 AT、TCC、Saga 和 XA 四种事务模式，在微服务架构中被广泛使用。

二、CAP 理论与 BASE 理论

1. CAP 理论

CAP 理论由 Eric Brewer 于 2000 年提出，是分布式系统的基石理论。它指出：一个分布式系统最多只能同时满足以下三个特性中的两个：

C（Consistency，一致性）：所有节点在同一时刻看到的数据完全相同。当数据写入一个节点后，任何从其他节点读取的请求都能获得最新写入的值。

A（Availability，可用性）：系统在任何时刻都能正常响应请求。即使部分节点故障，系统仍能对外提供服务，返回非错误的响应（不保证数据最新）。

P（Partition Tolerance，分区容错性）：当网络发生分区（节点间通信中断）时，系统仍能继续运行。在分布式环境中，网络分区是不可避免的（网络延迟、丢包、断连），所以 P 是必须满足的。

由于 P（分区容错性）在分布式系统中必须保证，因此实际选择落在 C 和 A 之间：

CP 系统（牺牲可用性）：当网络分区发生时，系统优先保证数据一致性，可能拒绝服务或返回错误。典型代表：ZooKeeper、Etcd、HBase。

AP 系统（牺牲一致性）：当网络分区发生时，系统优先保证可用性，但可能返回旧数

据，后续通过异步方式同步。典型代表：Eureka、Cassandra、DynamoDB。

CA 系统在分布式环境中不存在，因为网络分区无法避免。对于微服务架构中的分布式事务，通常需要在一致性和可用性之间做权衡，不能同时完美满足两者。

2. BASE 理论

BASE 理论是对 CAP 理论中 AP 的延伸和补充，是分布式事务设计的实践指导原则。它的核心思想是：即使无法做到强一致性，应用也可以采用适当的方式达到最终一致性。BASE 由以下三个概念组成：

BA（Basically Available，基本可用）：分布式系统出现故障时，允许损失部分可用性，但核心功能仍然可用。例如：电商大促时，系统可能降级（关闭非核心功能、返回缓存数据），但核心下单流程仍能工作。

S（Soft State，软状态）：系统中的数据允许存在中间状态，且该中间状态不影响系统的整体可用性。即允许不同节点之间的数据副本存在短暂的不一致。

E（Eventually Consistent，最终一致性）：系统中的所有数据副本，经过一段时间的同步后，最终能够达到一致的状态。不要求实时一致，但保证最终会一致。

BASE 理论与 ACID 的对比：

ACID（传统单机事务）：追求强一致性，采用悲观锁机制，适用于银行转账等对一致性要求极高的场景。

BASE（分布式事务）：接受最终一致性，采用乐观机制，适用于电商、社交等对可用性要求高、可容忍短暂不一致的场景。

在微服务架构的分布式事务实践中，BASE 理论是 Seata AT 模式、Saga 模式等方案的理论基础。

三、什么是 XA 模式？什么是 AT 模式？两者的优劣

1. XA 模式

XA 是 X/Open 组织定义的分布式事务处理标准，基于两阶段提交协议（2PC）实现。在 XA 模式中，事务管理器（TM）与资源管理器（RM，即数据库）通过 XA 接口进行通信。

工作流程（两阶段提交）：

第一阶段 Prepare（预提交）：事务管理器向所有参与的数据库发送 prepare 指令。每个数据库执行 SQL 操作，将 undo 和 redo 日志写入磁盘，但暂不提交事务。完成后回复 ready 或 fail。

第二阶段 Commit/Rollback（提交/回滚）：如果所有数据库都返回 ready，事务管理器发送 commit 指令，所有数据库提交事务。如果任一数据库返回 fail，事务管理器发送 rollback 指令，所有数据库回滚事务。

XA 模式优点：

强一致性：严格遵循 ACID 特性，事务要么全部提交、要么全部回滚，数据不会出现中间状态。

标准化：遵循 X/Open 国际标准，主流数据库（MySQL、Oracle、PostgreSQL 等）原生

支持，无需额外改造。

对业务无侵入：事务逻辑由数据库和事务管理器处理，业务代码无需关注分布式事务细节。

XA 模式缺点：

性能差：Prepare 阶段所有数据库需要锁定资源并等待，锁定期间其他事务无法操作相关数据，高并发场景下吞吐量严重下降。

单点故障：事务管理器是中心化节点，一旦宕机，所有处于 Prepare 状态的资源将被无限期锁定，即悬挂事务问题。

阻塞协议：协调者故障会导致参与者长时间阻塞，系统可用性下降。

数据库依赖：要求所有参与方数据库都支持 XA 协议，混合数据库环境下可能受限。

2. AT 模式

AT 模式是 Seata 独创的分布式事务解决方案，属于补偿型事务。它基于两阶段协议，但和 XA 模式有本质区别：AT 模式在第一阶段就直接提交事务，不锁定数据库资源；如果全局事务最终需要回滚，则通过第二阶段的反向补偿（执行反向 SQL）来恢复数据。

工作流程（两阶段）：

第一阶段 业务数据 + 回滚日志：执行实际的业务 SQL 操作，将数据修改提交到数据库（直接 commit）。同时，Seata 自动记录回滚日志（undo_log 表），保存修改前后的数据快照。事务提交后，数据库锁立即释放。

第二阶段 提交或回滚：如果全局事务提交，异步删除 undo_log 记录。如果全局事务回滚，根据 undo_log 记录生成反向补偿 SQL（INSERT 转 DELETE，UPDATE 转反向 UPDATE，DELETE 转 INSERT），执行后将数据恢复到修改前的状态。

AT 模式核心机制——全局锁：在第一阶段提交本地事务前，Seata 会获取该行数据的全局锁，确保在全局事务提交前，其他事务不能修改该数据，防止脏写问题。全局锁存储在 Seata Server（TC）端，本地事务提交后会立即释放数据库行锁，但保留全局锁直到全局事务结束。

AT 模式优点：

高性能：第一阶段直接提交事务，数据库锁立即释放，不长时间占用数据库资源。高并发场景下吞吐量远高于 XA 模式。

无阻塞：本地事务快速提交，不会像 XA 那样长时间阻塞其他事务。

业务无侵入：Seata 通过数据源代理（DataSourceProxy）自动生成回滚日志和执行补偿 SQL，业务代码无需任何改动。

最终一致性：遵循 BASE 理论，数据在短时间内一致，适合绝大多数互联网业务场景。

AT 模式缺点：

弱一致性：第一阶段提交后、第二阶段回滚前存在中间状态，数据短暂不一致，对强一致性场景不适用（如金融记账）。

额外存储开销：需要 undo_log 表保存回滚日志，以及 Seata Server（TC）存储全局事务状态和全局锁信息。

SQL 支持有限：仅支持 INSERT、UPDATE、DELETE 操作的自动补偿，复杂 SQL（如存储

过程、DDL) 不适用。

依赖 Seata Server: 需要额外部署和维护 Seata Server (TC), 增加了运维复杂度。

3. XA 模式 vs AT 模式 对比总结

一致性: XA 强一致性 (ACID), AT 最终一致性 (BASE)。XA 更适合金融、支付等对数据一致性要求极高的场景。

性能: AT 远优于 XA。XA 在两阶段期间一直持有数据库锁, AT 在第一阶段提交后即释放。高并发场景下 AT 的吞吐量可达 XA 的数倍。

可用性: AT 优于 XA。XA 的协调者故障会导致资源被无限期锁定, AT 无此问题, 本地事务已提交不会阻塞。

侵入性: XA 依赖数据库支持 XA 协议; AT 依赖 Seata 代理数据源, 业务代码均无侵入。

适用场景: XA 适用于银行、证券等强一致性场景, 但性能开销大。AT 适用于电商、社交等互联网业务, 在性能和一致性之间取得了良好平衡, 是微服务架构中最为常用的分布式事务方案。